

Chapter 2: Supervised Learning

Course material

Universidad de Salamanca

Faculties of Sciences and Chemical Sciences

September 24, 2026

In this chapter

- ① **2.1 Regression:** predicting a continuous value
 - Linear regression, least squares, gradient descent
 - Regularization: Ridge, Lasso, Elastic Net
- ② **2.2 Classification:** predicting a category
 - Logistic regression, trees, Random Forest, SVM
 - Ensembles, neural networks, and how to evaluate them

What is supervised learning?

- We have **data** (input features, X) and **labels** (correct answers, y)
- The goal is to learn the relationship $X \rightarrow y$
- It's like learning with a “supervisor” who corrects your answers during training

Regression (continuous value)

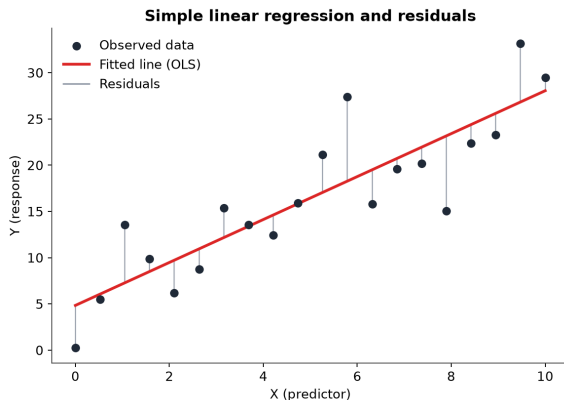
- House price
- Tomorrow's temperature

Classification (category)

- Is this email spam?
- Is this image a cat, dog, or bird?

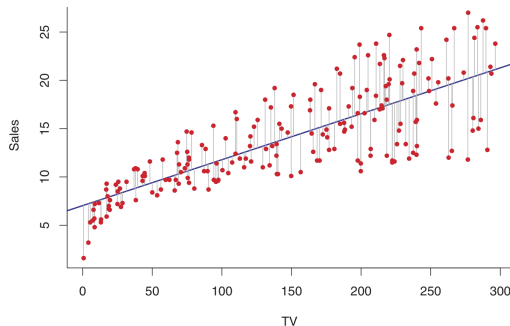
Linear regression

- **Simple:** a single predictor, $Y \approx \beta_0 + \beta_1 X$
- **Multiple:** one coefficient per feature, $Y = \beta_0 + \beta_1 X_1 + \dots + \beta_p X_p + \epsilon$
- β_0 (intercept) and $\beta_1 \dots \beta_p$ (weights) are what the model learns from the data
- In p dimensions, the model represents a **hyperplane**



Real example: the *Advertising* dataset

- Predicts *Sales* from TV, radio, and newspaper advertising spend
- The line minimizes the sum of squared residuals (OLS)



Source: James, Witten, Hastie & Tibshirani (2013), *An Introduction to Statistical Learning*, Fig. 3.1. Freely distributed textbook (statlearning.com).

Ordinary Least Squares (OLS)

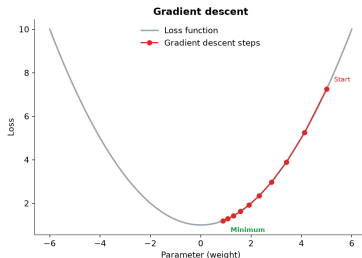
- A **residual** $e_i = y_i - \hat{y}_i$ is the difference between the real and predicted value
- OLS finds the weights that **minimize** the sum of squared residuals: $RSS = \sum_{i=1}^n (y_i - \hat{y}_i)^2$
- It has an **exact analytical solution**: $\hat{\beta} = (X^T X)^{-1} X^T y$
- Geometric view: $\hat{y}$ is the orthogonal projection of y onto the space spanned by X

Example: with just 3 houses (m^2 and price), OLS finds in a single step the line that best summarizes that relationship — no trial and error needed.

Gradient descent

- When data is massive, solving the exact equation is too costly
- **Iterative** algorithm: starts from random weights and nudges them toward lower error, step by step
- $\theta \leftarrow \theta - \alpha \nabla J(\theta)$, where α is the **learning rate**
- Variants: *Batch* (whole dataset), *Stochastic* (one sample), *Mini-batch* (a small group)

Example: it's like walking down a foggy mountain: at each step you feel the slope under your feet and move toward where it descends most, until you reach the valley.

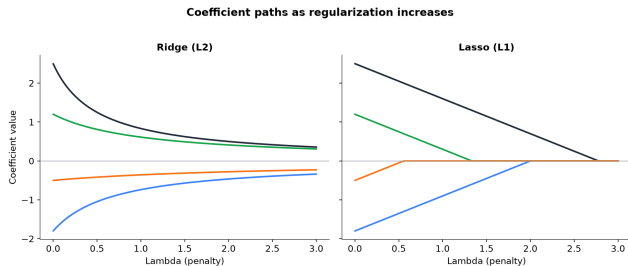


Polynomial regression and the risk of overfitting

- Powers of the original predictors (X^2 , X^3 ...) are added to capture curves
- Still a model that is **linear in its parameters** — can still be trained with OLS
- The higher the degree, the higher the risk of memorizing noise (overfitting)

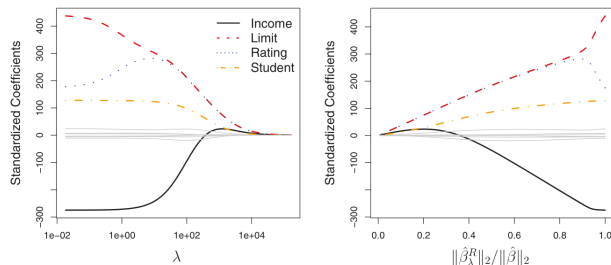
Regularization: Ridge and Lasso

- Penalize large weights in the cost function, forcing simpler models
- **Ridge** (L_2): penalizes $\sum \beta_j^2$ — shrinks coefficients without zeroing them
- **Lasso** (L_1): penalizes $\sum |\beta_j|$ — can push coefficients exactly to zero (variable selection)
- **Elastic Net**: combines both penalties



Ridge on real data: the *Credit* dataset

- As λ increases, coefficients shrink toward zero
- Trades a small increase in bias for a reduction in variance



Source: James, Witten, Hastie & Tibshirani (2013), *An Introduction to Statistical Learning*, Fig. 6.4.

Validation and diagnostics in regression

- **MSE / RMSE**: mean squared error (and its root); heavily penalizes large errors
- **MAE**: mean absolute error; more robust to outliers
- R^2 : proportion of variance explained by the model (1 = perfect, 0 = as good as predicting the mean)
- **Validation curves**: plotting training and validation error against complexity reveals high bias (both errors high) or high variance (a gap between them)

Summary: 2.1 Regression

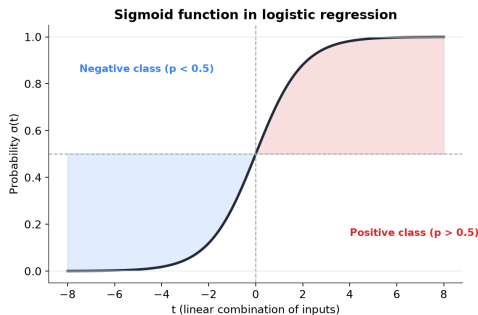
- **Linear regression**: simple yet very powerful
- **OLS**: exact solution; **gradient descent**: general and scalable
- **Polynomial regression**: captures non-linear relationships, at the cost of more overfitting risk
- **Regularization**: Ridge shrinks, Lasso selects variables

Classification: predicting categories

- The goal is to predict a **category**, not a continuous number
- Examples: spam vs. not-spam, sick vs. healthy, cat vs. dog vs. bird
- **Logistic regression** starts by estimating the **probability** of belonging to a class

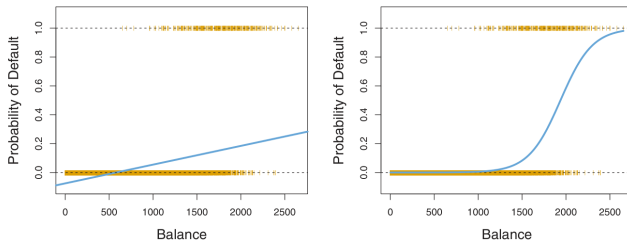
The sigmoid function

- Maps any real value to a probability between 0 and 1
- $\sigma(t) = \frac{1}{1 + e^{-t}}$, where t is the *logit* (a linear combination of the features)
- Trained by minimizing **cross-entropy** (log loss) with gradient descent



Why not use linear regression?

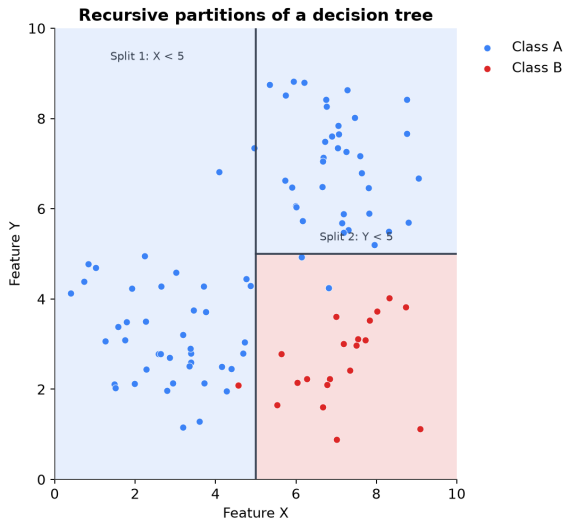
- Linear regression can predict probabilities < 0 or > 1 (nonsensical)
- Logistic regression uses the sigmoid to keep the output always within $[0, 1]$



Source: James, Witten, Hastie & Tibshirani (2013), *An Introduction to Statistical Learning*, Fig. 4.2.

Decision trees

- Split the feature space with successive orthogonal cuts, like a flowchart of yes/no questions
- **CART** algorithm: each split maximizes the purity of the child nodes (Gini index or entropy)
- Easy to interpret, but prone to overfitting without **pruning**

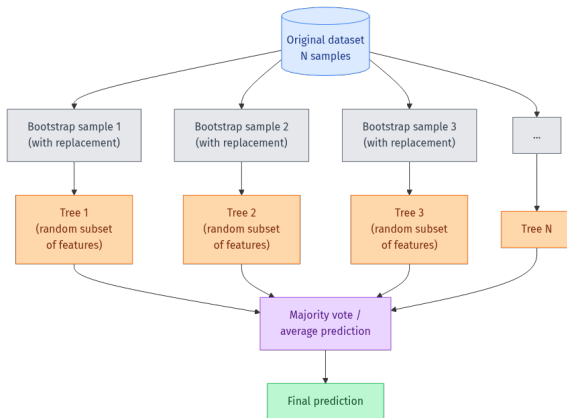


Example: “does the email contain the word ‘free’? → yes → does it have more than 3 links? → yes → spam.”

Random Forests

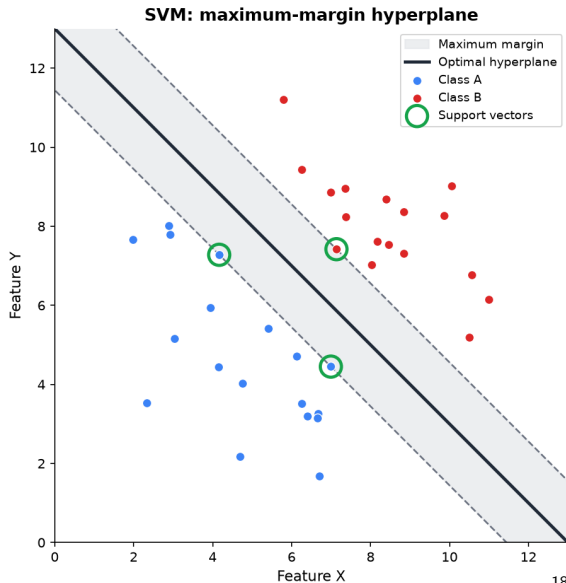
- Combine **many trees** to reduce variance and improve robustness
- **Bagging**: each tree is trained on a different *bootstrap* sample (with replacement)
- **Decorrelation**: only a random subset of features is considered at each split

Several bootstrap samples, a different tree per sample, and predictions combined by majority vote or averaging.



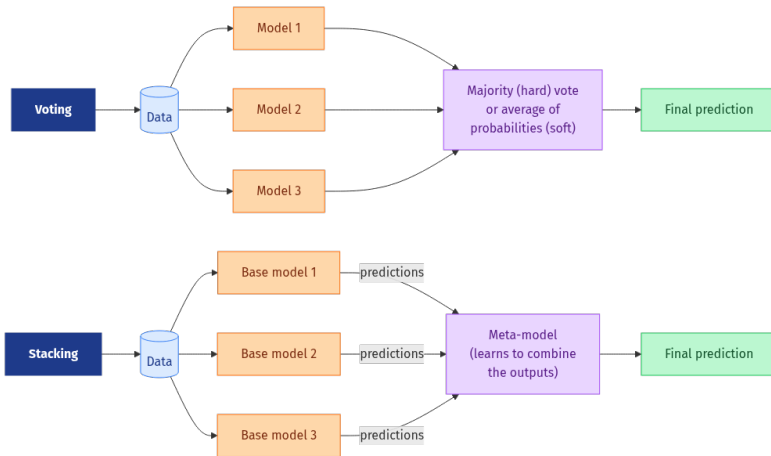
SVM: maximum margin

- Finds the **hyperplane** that separates the classes with the **maximum margin**
- **Support vectors** are the only points that determine the boundary
- The *kernel trick* enables non-linear boundaries, mapping the data into a higher-dimensional space



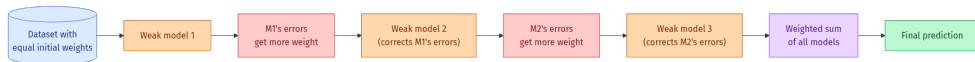
Voting and stacking

- Combine **several models** to get a better prediction than any single one
- **Voting**: majority vote or average of probabilities, with a fixed rule
- **Stacking**: a meta-model **learns** how to combine the base models' outputs



Boosting

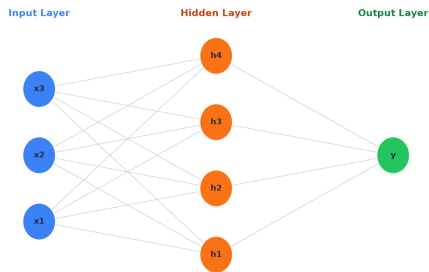
- Unlike the parallel training of random forests, boosting trains models **sequentially**
- Each new model tries to correct the previous one's errors, weighting misclassified instances more heavily
- Modern examples: XGBoost, Gradient Boosting



Neural networks

- Layers of neurons connected by adjustable weights
- Each neuron applies a non-linear activation function: **ReLU** in hidden layers, **Softmax** in the output layer
- **Backpropagation**: propagates the error backward from the output to adjust every weight

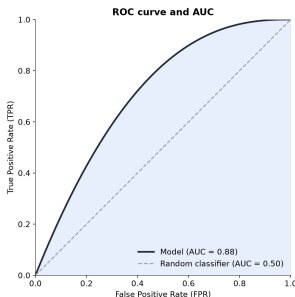
Architecture of a fully-connected neural network



Evaluating classifiers

- **Accuracy**: can be misleading when classes are imbalanced
- **Precision** and **recall**: how reliable are the positives vs. how many real positives are found
- **F1**: a balance between both
- **ROC-AUC**: compares models without depending on a fixed threshold

Example: if 99% of transactions aren't fraud, a model that always says "not fraud" would have 99% accuracy without having learned anything useful.



Training and diagnostic strategies

- **Beat a baseline:** before going complex, check you beat a trivial (random) classifier
- **Learning curves:** training/validation loss and accuracy reveal the exact point of overfitting
- **Early stopping:** stop training once validation stops improving
- **Error analysis:** manually inspect the cases where the model fails

Summary: 2.2 Classification

- **Logistic regression**: extends linear regression to probabilities
- **Trees**: interpretable, prone to overfitting; **Random Forest** fixes it by combining many
- **SVM**: optimal separation, with the kernel trick for complex boundaries
- **Ensembles** (voting, stacking, boosting): combine models to improve performance
- **Neural networks**: powerful but harder to interpret

End of Chapter 2

Next: Chapter 3 — Unsupervised Learning